CS781 Capstone Project: In-House Legal Intelligence System

Soham Girish Panchal
231014
sohamgp23@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

Law firms cannot send confidential files to external APIs, so this project builds a self-hosted legal NLP stack from scratch under a 50M–300M parameter cap using only the official course data. Phase 1 pre-trains four 125M-parameter models (GQA decoder, latent-attention decoder, encoder–decoder, RetNet-style retention decoder) on 200M tokens drawn equally from Supreme Court, High Court, Tribunal and Acts text, with a byte-level BPE trained from scratch. Every later phase (NER, rhetorical roles, judgment prediction, statute identification, summarization, prior-case retrieval) starts from these checkpoints and compares 6–8 adaptation methods written from scratch, including LoRMA and knowledge distillation. Decoding and length control mattered more than perplexity, dev-tuned decision rules gave the largest gains in classification, partial fine-tuning beat every adapter on strict NER (locally), students distilled from ensembles matched their teachers, and a tuned BM25 with a temporal prior beat all dense retrievers.

1 Competition Result

Codabench username: s_231014 **Roll number:** 231014
GitHub repository: https://github.com/Sohamp05/CS781-Capstone-Project

Table 1 lists the best valid score per phase and the rank of that score on the public Codabench leaderboard (rank / number of ranked participants).

Table 1: Official phase scores and public-leaderboard ranks, recorded on 9 November 2026.

Phase	Task	Primary metric (value)	Rank	Valid submissions
1	Legal LM Pretraining	BERTScore-F1 0.8317	64 / 142	6
2	Legal NER	Strict macro-F1 0.3720	112 / 133	6
3	Rhetorical role prediction	Macro-F1 0.5075	55 / 132	8
4	Court judgment prediction	Macro-F1 0.6934	81 / 132	8
5	Legal statute identification	Macro-F1 0.3635	81 / 133	8
6	Abstractive summarization	BERTScore-F1 0.8350	83 / 131	10
7	Prior Case Retrieval (PCR)	Micro-F1@7 0.4076	59 / 133	9

Total valid Codabench submissions across all phases: 55 scored (63 uploads; 7 failed or were cancelled on the server and 1 never left the queue) **Date ranks were recorded:** 9 November 2026

Every phase has at least four valid submissions (six to ten), and Phases 2–7 each include at least three adaptation techniques (LoRMA among them) plus distillation on more than one Phase-1 model; Phase-2 partial and full fine-tuning finished after the deadline and are reported locally only.

2 Introduction and Problem Description

A law firm that wants entity extraction, outcome prediction, statute tagging, summarization or precedent search cannot send confidential judgments to a third-party model, so the system must be trained and served in-house on modest hardware. The capstone poses seven tasks on Indian legal text, each with a fixed input/output schema: (1) a 512-word prompt → a 128-word continuation (BERTScore-F1 [1]; ROUGE-L [2], BLEU-4 [3]); (2) a judgment → `[start, end, label]` character spans over 12 types (strict macro-F1); (3) a judgment's sentence list → one of 13 roles per sentence (macro-F1); (4) a judgment → a 0/1 outcome (macro-F1); (5) a fact description → a set of statute IDs out of 100 (macro-F1); (6) a judgment → a free-text summary (BERTScore-F1); (7) a query judgment and a candidate pool → 7 ranked candidate IDs (micro-F1@7). The foundation is trained from scratch (50M–300M parameters, official data only) on free Kaggle T4s, and every later phase must start from it. The questions studied are architecture choice under a fixed 200M-token budget, decoding versus perplexity, which from-scratch adaptation methods transfer best, long-document reading, dev-tuned decision rules and distillation. Built end to end: the tokenizer, four backbones, every adaptation method, BM25 and the ensembles. Deliberately not attempted: pre-training beyond 200M tokens, continued pre-training on task text, alignment or post-training, feeding Phase-2/3 outputs into later phases, cross-encoder re-ranking and nested NER.

3 Data Analysis and Preprocessing

3.1 Sources and official splits

Only the official `Exploration-Lab/CS-781-Capstone` data, starter code and evaluators [4] were used (Table 2). Phases 2 and 6 have no dev set, so 10 of 70 NER documents and 300 summarization documents (ID hash; 120 scored) were held out.

3.2 Corpus statistics that actually informed design

Phase 1. The sources differ in size, document length (Table 2, Fig. 1d, from 25 MB samples) and register (enumerated statutory text versus citation-dense narrative reasoning). Raw proportions would be dominated by High Courts, so an equal 25% token share was taken from each source. All four architectures were first trained and submitted on a Supreme-Court-heavy mix (0.50/0.30/0.10/0.10), then re-trained on the equal mix so that all four backbones share one tokenizer, training stream and validation set (fair comparison, any teacher–student pairing for distillation); Sec. 5.3 gives the effect. **Phase 2.** Hold-out support ranges from 324 (DATE) to 28 (WIT); PREC and CASENO spans are long and punctuation-heavy, while COURT and DATE recur in every letterhead. **Phase 4.** The court's reasoning and order sit at the end of a judgment, so the model reads the tail; the train/dev label skew means a 0.5 threshold learned on train is wrong on test. **Phase 5.** No statute is rare ($\geq$272 documents). **Phase 6.** Summaries are long and largely extractive: copying the first N words of the judgment (N from a length model) already gives ROUGE-1 0.507 and BERTScore 0.828 on the hold-out set. **Phase 7.** The files carry only IDs and text, no court metadata, so court-level structure was not used; pools overlap across splits (740 test candidates are also in the training pool). Only 8.8% of training gold pairs cite a candidate whose judgment date is later than the query's, against 37.5% of all pairs, which motivated a temporal penalty.

Table 2: Splits and design-relevant statistics (words are whitespace tokens).

Ph.	Train	Dev / held-out	Test	Statistics that informed the design
1	202.6M tokens (≈50M each: SC, HC, Trib., Acts)	4.37M tokens (2% of docs)	1,000 prompts	Raw files: SC 1.32 GB, HC 10.67 GB, Tribunals 6.48 GB, Acts 0.73 GB. Median document length in a 25 MB sample of each file: 3,752 / 258 / 3,382 / 2,835 words (p90 10.3k / 1.0k / 9.6k / 12.6k), so High Court documents are mostly short orders. 1.22 tokens/word; prompts average 609 tokens (max 916), so 2,048 tokens hold prompt and continuation.
2	70 docs, 460.6k tokens, 8,393 entities	10 docs	35 docs	DATE 1,575, STAT 1,260, PREC 1,183, COURT 1,013, AUTH 1,003, CASENO 760, APP 407, RESP 381, JUDGE 245, A.COUNSEL 195, WIT 187, R.COUNSEL 184; span length 1 word 30.7%, 2–3 48.7%, 4–7 14.2%, 8+ 6.4% (median 2; PREC 5, CASENO 4; max 35); no nested or overlapping gold spans (146 adjacent pairs); 98.2% on BPE boundaries, oracle strict macro-F1 0.987.
3	CL 40 docs / 10,593 sent., IT 40 / 6,252	CL 5 / 1,396, IT 5 / 746	10 docs, 2,197 sentences	13 roles (Fig. 1a); median sentence 26–28 words (p90 58–67), median sentences per document 123 (CL) and 129 (IT) in train (means 265 and 156); Dissent occurs only in CL train (284, none in dev), PrecedentOverruled 3 times.
4	37,387 docs (32,305 unique IDs)	3,505 (2,511 unique)	1,517	Train is 58.6% class 0, dev and test are balanced (50.1%, 50.2% class 1); median 2,035 words, p90 6,450.
5	42,750 docs	10,181	13,019	100 statutes, 3.70 labels/doc (max 31); every statute has ≥272 training docs; median 430 words; 25% exceed 2,048 tokens.
6	7,030 pairs	300 (120 scored)	100	Judgment median 3,127 words, summary 640; compression 0.22; only 8.4% of summary words are absent from the judgment.
7	827 queries, pool 4,320	118 q., pool 1,023	237 q., pool 1,727	Median query 4,508 words, candidate 4,036 (dev) / 3,947 (test), p90 13.6k–16.6k; 6.4 gold cases per query, 30% have more than 7 (oracle F1@7 0.64 train, 0.62 dev; Fig. 1b).

3.3 Preprocessing

Tokenization. A 32k byte-level BPE [5] was trained on a 200M-character sample of all four sources (no lowercasing; punctuation and Unicode kept). It is lossless and returns character offsets, unlike the assignments' `Whitespace` pre-tokenizer. Runs of whitespace are collapsed because the test prompts are plain strings. Citations such as "(2003) 4 SCC 481" and section symbols are split into frequent sub-words; no special tokens or handling were added for them or for party names. **Offsets for L-NER.** Whitespace normalization shifts positions, so the pipeline keeps a map $n2o$ from each normalized character to its original index and its inverse $o2n$. Gold spans are moved into normalized space; every BPE token inside an entity gets a B- or I- tag (25 tags); predicted runs are mapped back through $n2o$ (end = $n2o$[last char] + 1, after trimming the leading space that byte-level BPE attaches). Before training, the notebook encodes the gold spans, decodes them and scores the result with the strict metric. The oracle macro-F1 of 0.987 shows that the pipeline loses almost nothing (1.8% of entities start or end inside a token); this check is what prevented a "silent zero". **Long documents.** Phase 3 uses the provided sentence lists unchanged, so sentence counts always match (sentences over 128 tokens keep head and tail). Phase 4 keeps the last 4×512 tokens; Phase 5 the first chunks plus the final one (up to 4×512); Phase 6 decoders read the first 960 and last 192 tokens, the encoder–decoder up to four 1,024-token chunks. In Phase 7 query and candidate share one encoder and one policy: 16 evenly spaced 512-token chunks are stored per document and 8 are read at inference, so the whole judgment is sampled.

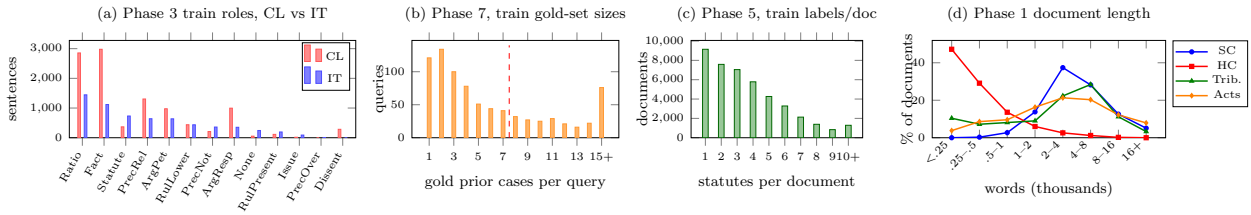


Figure 1: (a) Roles are imbalanced and their mix differs by domain (Dissent only in CL), motivating class weighting, a domain embedding and dev-tuned class biases. (b) Queries with more than 7 gold cases (right of the dashed line, 30%) cap recall and those with 1–3 cap precision, so the best achievable F1@7 is 0.64. (c) 42,750 Phase-5 documents carry 1–31 statutes (mean 3.70); 21% have one, 8% have eight or more. (d) Phase-1 document length per source, from a 25 MB sample at the start of each file: High Court documents are mostly short orders.

Table 3: Phase-1 architectures ($d = 768$, 12 heads, FFN 2,048, 200M tokens, equal mix). Local BERTScore: 48 held-out prompts, best decoding. †Codabench score of the first-round SC-heavy run (not re-submitted).

Arch.	Sequence mixer	Layers	Params	Val loss (ppl)	Local BERTScore	Train time	Shortest output	Codabench
GQA	causal GQA 12Q/4KV + RoPE	16	125.3M	**2.724** (15.2)	0.8485	5.9 h	2 words	0.8307†
MLA	latent KV (rank 192) + decoupled RoPE	16	125.7M	2.810 (16.6)	**0.8498**	5.5 h	51 words	**0.8317**
Enc-dec	bidirectional enc. + causal dec.	6+6	123.7M	3.163 (23.6)	0.8175	2.5 h	32 words	0.8148
RetNet	multi-scale retention	13	124.3M	3.090 (22.0)	0.8486	7.1 h	118 words	0.8286†

4 Approach and Model Evolution

4.1 Phase-1 foundation model

All four Transformer-style [6] architectures share $d = 768$, 12 heads, a SwiGLU [7] FFN of 2,048, pre-RMSNorm [8], tied embeddings and the 32k vocabulary, and differ only in the sequence mixer (Table 3; the best model is drawn in Fig. 2). **GQA** is a LLaMA-style decoder with grouped-query attention [9] (12 query heads share 4 KV heads) and RoPE [10]. **MLA** rebuilds all keys and values from one 192-d latent with a decoupled 16-d rotary key, as in DeepSeek-V2 [11]. **Enc-dec** is T5-style [12] (6 bidirectional encoder and 6 causal decoder layers with cross-attention), trained as a prefix LM: a 1,280-token window is split at a random point (512–1,024 encoder tokens) and the decoder predicts the rest, which mirrors the task. **RetNet** replaces softmax attention by multi-scale retention [13], $S_n = \gamma_h S_{n-1} + k_n^\top v_n$, $o_n = q_n S_n$ with per-head decay $\gamma_h = 1 - 2^{-5-h}$, a swish gate and per-head RMS norm; training uses the exact chunkwise form (chunk 256, fp32) and generation carries a constant-size state; 13 layers match the parameter count. Each model asserts $50\text{M} \le \#\theta \le 300\text{M}$ and passes unit tests on a tiny copy (causality, encoder bidirectionality, incremental decoding equal to the full forward pass). Training uses next-token cross-entropy on 2,049-token blocks, one pass over 200M tokens (3,050 steps of 65.6k tokens), AdamW [14] ($\beta = (0.9, 0.95)$, weight decay 0.1), peak LR 5×10^{-4} with 150 warm-up steps and cosine decay to 10%, fp16 and clipping at 1.0. Because the encoder–decoder is only trained on its decoder half, models are compared by a shared *continuation loss*: the NLL of 512 tokens given a 768-token prefix of held-out documents. Submissions use KV-cache or state decoding with special tokens masked, a repetition penalty on generated tokens [15] and nucleus sampling [16]; generation runs until every sequence has 128 words and is cut to exactly 128. Each model's decoding was picked from a 5-setting sweep on 48 held-out prompts. *What failed first.* The first uploads failed on the server, and the first GQA submission had 3 continuations under 128 words (two looping), which led to the length-aware stopping rule and to repetition penalties and n-gram blocking later.

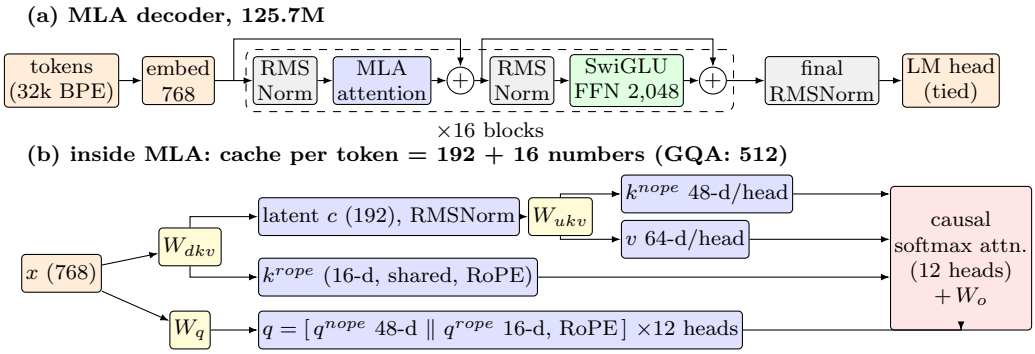


Figure 2: Best Phase-1 model (MLA, equal mix; 0.8317): all heads' keys and values come from one latent; RoPE via a shared key.

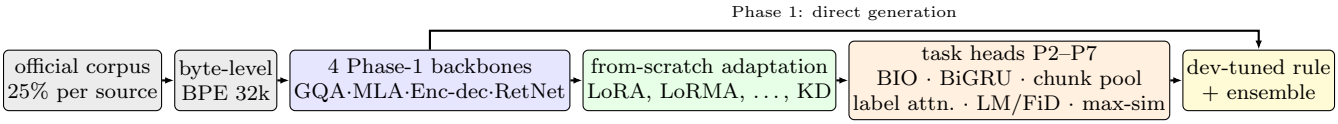


Figure 3: System overview: one tokenizer and four from-scratch backbones; each later phase adds an adaptation method, a task head and a dev-tuned decision rule or ensemble.

4.2 Transfer protocol (Phases 2–7)

Every task model descends from one Phase-1 export (weights, config, model code, tokenizer): the LM head is dropped (and, except in Phase 6, the enc-dec decoder), an adaptation method is applied and a task head is trained (Fig. 3). Decoders are causal, which suits token and sentence labelling poorly, so in Phases 2–3 each window is fed twice, $[w, w]$, and the head reads the second copy, which sees the whole window through the first. All methods are implemented from scratch (no `peft`/`bitsandbytes`): LoRA [17], $W = W_0 + \frac{\alpha}{r}BA$; the compulsory LoRMA+ [18], $W = (I + \frac{\alpha}{r}BA)W_0$, computed as $h_0 = W_0x$, $h = h_0 + \frac{\alpha}{r}B(Ah_0)$ with $B = 0$ at initialisation; rsLoRA [19]; AdaLoRA [20] (SVD form with importance-based rank pruning); IA^3 [21]; QLoRA [22] with a custom blockwise NF4 (RetNet weights 497 MB→56 MB); Houlsby [23] and Pfeiffer adapters (the single-adapter configuration of [24]); BitFit [25]; prompt tuning [26]; partial (top 6 blocks) and full fine-tuning. Merged LoRA/LoRMA weights reproduce the unmerged logits (max. difference 1.8×10^{-5} and 4.3×10^{-5}). Distillation [27] uses a student built from a Phase-1 model by keeping every other layer, initialised from its weights and fully fine-tuned on $a\,\text{CE} + (1-a)\,T^2\,\text{KL}(p_T^{(T)} \| p_S^{(T)})$; from Phase 3 on the teacher is the mean prediction of the strongest earlier runs. Every phase uses all four backbones.

4.3 Phase 2 - Legal NER

The model uses token classification with offset reconstruction (Sec. 3.3) rather than direct span prediction, because the tokenizer gives exact offsets and the oracle proved the round trip. Documents (6.6k tokens on average) are cut into 384-token windows (512 for the encoder) with stride 192; at inference, overlapping windows' log-probabilities are averaged with a triangular weight that trusts window centres most, so an entity cut by one window is seen whole by the next. Decoding takes the arg-max tag per token and merges BIO runs, which also resolves overlaps (BIO cannot represent nested spans, so none are produced); there is no thresholding or n-best decoding. The 12 types are APP (appellant/petitioner), RESP (respondent), A.COUNSEL, R.COUNSEL, JUDGE, WIT (witness), AUTH (authority), COURT, STAT (statute), PREC (precedent), DATE and CASENO (case number). A linear head over 25 tags is trained with cross-entropy; later runs down-weight

the `O` tag to 0.5 against imbalance. Runs: LoRA ($r = 16$, $\alpha = 32$) on GQA, LoRMA+ ($r = 16$) on RetNet, Houlsby adapters ($m = 64$) on MLA, distillation from LoRA-GQA into a 6-of-13-layer RetNet ($a = 0.5$, $T = 2$), and, after the phase closed, partial fine-tuning of the top 6 GQA blocks and full fine-tuning of the enc-dec encoder.

4.4 Phase 3 - Rhetorical role prediction

Consecutive sentences are packed into $\leq$320-token chunks, so each sentence is encoded together with the neighbouring sentences in its chunk (sentences over 128 tokens keep head and tail). Each sentence is pooled (masked mean plus attention pooling), a CL/IT domain embedding and relative-position features are added, and a 2-layer BiGRU over all sentences of the document provides document-level context before a linear layer over 13 roles; no Phase-2 entities are used as features. One joint model covers both domains (the domain embedding absorbs the shift). Training uses class-weighted cross-entropy (inverse frequency$^{0.5}$), label smoothing 0.05 and early stopping on dev macro-F1; a CRF was implemented but not used. Per-class logit biases tuned on dev helped only the ensemble in a 2-fold check, so single runs are submitted without them. The backbone is frozen in all PEFT runs: LoRMA (GQA), IA^3 (MLA), AdaLoRA (r 12$\rightarrow$6) and rsLoRA ($r = 64$) (RetNet), LoRA (enc-dec encoder), distillation into an 8-layer GQA, a sentence-only ablation and a greedy ensemble.

4.5 Phase 4 - Court judgment prediction

The last 4×512 tokens are split into chunks from the end; each chunk is encoded, pooled (mean plus attention) and given a position-from-end embedding, and chunk vectors are combined by attention pooling together with the final-chunk vector into two logits. Training samples 2–4 tail chunks per batch and 10k random documents per epoch with class-weighted cross-entropy, under a 75-minute budget. Since train is skewed and test balanced, the threshold on $p(1)$ is tuned on dev and kept only if a 2-fold check on dev confirms it. Runs: full FT and LoRMA (GQA), BitFit (MLA), partial FT and AdaLoRA (RetNet), Houlsby adapters (enc-dec), distillation into an 8-layer GQA from the mean of six runs, and a greedy ensemble of mean $p(1)$. Phase-2/3 outputs are not used as features.

4.6 Phase 5 - Legal statute identification

Up to four chunks (first ones plus the final one) are encoded, projected to 384 dimensions and given chunk-position embeddings. Two heads are summed: label-wise attention [28], where each statute's query attends over all tokens, and a global document vector. The loss is BCE with positive weight $\mathrm{clip}(\sqrt{\mathrm{neg}/\mathrm{pos}}, 1, 15)$ per statute and biases initialised at the label prior. A repeated 2-fold check on dev compares a fixed 0.5, one global threshold and per-statute thresholds shrunk towards the global one; per-statute thresholds won for every run, and every document gets at least one statute. Runs: the six Phase-4 techniques, distillation, a tail-only ablation and two ensembles.

4.7 Phase 6 - Abstractive summarization

Decoders continue `[head ... tail] + ''Summary of the judgment:''` with loss on summary tokens only (1,152 document and up to 880 summary tokens). The encoder–decoder uses fusion-in-decoder [29]: up to four 1,024-token chunks are encoded separately (no unseen positions) and the decoder attends to all 4,096 states, reading 3.5$\times$ more text. Generation is greedy with n-gram blocking and an optional repetition penalty; a length model fitted on train, $\ln w_{\mathrm{sum}} = 0.610 \ln w_{\mathrm{doc}} + 1.550$, sets a per-document length window, and three decoding settings are compared per run on

Table 4: Hyperparameters of the model behind each phase's best submission (for ensembles, the strongest member).

	P1 MLA	P2 LoRA-GQA	P3 AdaLoRA-RetNet	P4 full-GQA	P5 full-GQA	P6 LoRA-GQA	P7 full-GQA
Params (trainable), M	125.7 (all)	128.8 (3.49)	130 (5.7)	126.3 (all)	125.7 (all)	132.2 (6.95)	125.5 (all)
Context (tokens)	2,048	384 (×2)	320/chunk	4×512 tail	4×512	1,152+880	8×512
Peak LR (body / head)	5e-4	3e-4 / 1.5e-3	1e-3 / 1e-3	2e-5 / 1e-3	2e-5 / 1e-3	3e-4	2e-5 / 1e-3
Batch × accum.	4×8	8×2	96 sent. ×4	4×4	4×4	2×8	6 q. + 2 neg.
Epochs / steps	1 / 3,050	8 / 992	≤12	3×10k docs	4 (≤75 min)	≤4 ep. (≈440 steps/ep., ≤60 min)	≤8 ep. (≤40 min)
Dropout	0.0	0.05 adapter / 0.1 head	0.2	0.15	0.15	0.0 (no head dropout)	0.0 (no head dropout)
Decoding / threshold	T0.7, p0.95, rep. 1.05	arg-max BIO	arg-max	$p(1)\geq$0.335	per-statute	greedy, 4-gram block	w=0.75, λ=2

held-out documents (BERTScore ties broken by ROUGE-1, since roberta-large truncates at 512 sub-words). Runs: LoRA ($r = 32$, GQA), QLoRA (RetNet), Pfeiffer and AdaLoRA (MLA), LoRMA and Houlsby (enc-dec), an 8-layer GQA student distilled from LoRA-GQA (token KL plus gold CE), a one-chunk LoRMA ablation, and a consensus (minimum-Bayes-risk [30]) ensemble that keeps, per document, the run summary with the highest mean ROUGE-1 against the others. No alignment or post-training was used.

4.8 Phase 7 - Prior Case Retrieval (PCR)

A from-scratch BM25 [31] tuned on dev (uni+bigrams, $k_1 = 1.5$, $b = 0.9$, log query weights) gives dev F1@7 0.360. The neural retriever is a shared-tower bi-encoder [32]: each chunk is mean-pooled and projected to a 256-d unit vector, and relevance is chunk max-sim [33] or the cosine of mean vectors (chosen on dev MAP). Training uses InfoNCE [34] with one gold case, two BM25 hard negatives and in-batch negatives (the query's other gold cases masked) and a learnable temperature, for 40 minutes. The final score is $w\,z(\text{dense}) + (1-w)\,z(\text{BM25}) - \lambda\,[\text{candidate dated after query}]$, with w, λ tuned on dev and checked on held-out dev halves. Runs: LoRMA (enc-dec encoder), QLoRA (RetNet), full FT and prompt tuning (GQA), IA^3 and partial FT (MLA), an 8-layer GQA distilled from the three best runs' similarities over each query's BM25 top-60, a random-negative ablation and ensembles. *Failed:* zero-shot Phase-1 states reached only F1@7 0.24–0.26 (decoders) and 0.13 (encoder), every dense model alone stayed at ≈0.30, below BM25, and a citation-popularity prior got zero weight.

5 Experiments and Implementation Details

5.1 Compute and training protocol

All runs used Kaggle's two T4 GPUs with fp16 autocast. Phase-1 pre-training took 2.5–7.1 h per model (atomic checkpoints every 250 steps for resumption); from Phase 4 on, one notebook ran all experiments in parallel on both GPUs with budgets of 75, 75, 60 and 40 minutes per run (Phases 4–7). Runs: 8 pre-training runs (4 architectures × 2 mixes), then 8, 7, 7, 8, 8 and 8 training runs plus 0, 1, 2, 2, 2 and 2 ensembles in Phases 2–7. Task runs use AdamW, linear warm-up over 5–6% of steps then linear decay, clipping 1.0, weight decay 0.01 and a higher head LR, and keep the best dev checkpoint (P2 strict macro-F1, P3–P5 macro-F1 at the tuned rule, P6 summary-token CE, P7 MAP; Phase 3 also stops after 4 epochs without improvement); Table 4 lists the final models.

5.2 Local evaluation

Held-out data came only from public files: a 2% document-level Phase-1 split (plus 48 prompt/continuation pairs), a 15% document split of the Phase-2 folds, the official dev sets in Phases 3–5 and 7, and a hashed 300-document Phase-6 hold-out. Official evaluators (or exact re-implementations) were

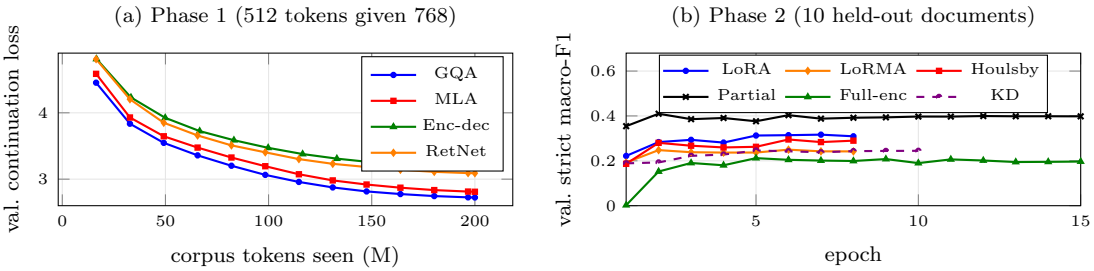


Figure 4: (a) GQA has the lowest held-out loss at every budget and the gaps stay roughly constant, suggesting that more training would not reorder the models. (b) Partial fine-tuning (30% of weights trainable) leads every PEFT method from the first epoch; the encoder-only model needs five epochs and stays lowest; LoRA/partial use GQA, LoRMA/KD RetNet, Houlsby MLA.

re-run locally, and every submission was checked for missing or unknown IDs, bad offsets or labels, duplicates and short continuations. Duplicated dev IDs (Phase 4: 2,511 unique of 3,505) and dev/train overlaps (Phases 5–6) slightly inflate local scores; local and Codabench scores still agree within ±0.03 in Phases 3–7, but not in Phase 2, where 10 held-out documents make macro-F1 noisy.

5.3 Ablations and controlled comparisons

Each comparison fixes the task model and budget and changes one factor. **Architecture (P1):** only the mixer changes (Table 3). **Decoding (P1):** GQA's local BERTScore is 0.830 greedy and 0.849 greedy with repetition penalty 1.15, a bigger gap than between the three decoders. **Context (P3):** removing neighbouring sentences and the document BiGRU from LoRMA-GQA drops dev macro-F1 from 0.494 to 0.348 and test from 0.499 to 0.380. **Threshold (P4):** the adapter model scores 0.336 at 0.5 and 0.569 with a threshold tuned on one half of dev and tested on the other. **Input region (P5):** for LoRMA-GQA, reading only the last four chunks instead of head plus tail lowers dev macro-F1 at per-statute thresholds tuned on the full dev set (not the honest 2-fold estimate of Table 5) from 0.3640 (head+tail) to 0.3618 (tail only). **Input length (P6):** one versus four encoder chunks gives the same BERTScore (0.8213). **Negatives (P7):** random instead of BM25 hard negatives gives a slightly *higher* dense MAP (0.308 vs 0.292) and the same hybrid F1@7. **Data mix (P1):** on test, the equal mix moves MLA from 0.8307 to 0.8317 and the encoder–decoder from 0.8201 to 0.8148 (SC-heavy vs equal). **Not run:** no task model was trained from random initialisation and the tokenizer was not compared with the `Whitespace` pre-tokenizer, so the gains from Phase-1 transfer and from the tokenizer are not measured directly (frozen Phase-1 states reach F1@7 0.245 in Phase 7 and 0.302 after training, which shows that fine-tuning helps, not that Phase-1 initialisation beats random initialisation).

6 Results

Table 5 lists every scored variant with its local score; Fig. 4 shows the Phase-1 validation curves and, as the representative downstream curve, Phase-2 dev macro-F1 per epoch (per-evaluation dev histories of Phases 3–7 are stored with each run in the repository).

Which system won and why. *Phase 1:* GQA had the lowest perplexity, yet MLA scored best: BERTScore barely separates the decoders but punishes short or looping outputs, which MLA avoided; the encoder–decoder trains on only its decoder half. *Phase 2:* only training more of the backbone helped: partial fine-tuning reached 0.410 local strict macro-F1 against 0.317 for

Table 5: Per-phase variants. Local: dev/held-out primary metric (P4–P5: honest 2-fold estimate at the tuned rule; P7: hybrid, honest). CB: official test primary metric; Sec.: official secondary metrics (Codabench only). Best per phase in bold; $*$ = scored only locally.

Variant (backbone)	Local	CB	Sec.
Phase 1 (Sec.: ROUGE-L / BLEU-4; SC = SC-heavy mix)			
MLA, equal mix	0.850	**0.8317**	.190 / .041
GQA, SC mix	0.842	0.8307	.1894 / .0471
MLA, SC mix	0.842	0.8307	.1890 / .0468
RetNet, SC mix	0.839	0.8286	.184 / .039
Enc-dec, SC mix	0.817	0.8201	.173 / .019
Enc-dec, equal mix	0.818	0.8148	.170 / .019
Phase 2 (Sec.: strict macro-P / R)			
LoRA $r16$ (GQA), first run	–	**0.3720**	.354 / .399
LoRA $r16$ (GQA), re-run	0.317	0.3632	.342 / .395
Houlsby $m64$ (MLA)	0.295	0.3381	.307 / .381
LoRMA+ $r16$ (RetNet), 2 runs	0.250	.3035/.2906	.268 / .359
KD LoRA-GQA → 6-layer RetNet	0.246	0.2912	.279 / .315
Partial FT top-6 (GQA)*	**0.410**	–	–
Full FT, enc-dec encoder*	0.212	–	–
Phase 3 (Sec.: macro-P / R)			
Ensemble (5 runs) + class biases	**0.509**	**0.5075**	.622 / .509
AdaLoRA 12→6 (RetNet)	0.485	0.5073	.575 / .521
KD ensemble → 8-layer GQA	0.472	0.5071	.518 / .509
LoRMA+ $r16$ (GQA)	0.494	0.4992	.569 / .510
IA^3 (MLA)	0.488	0.4893	.555 / .500
rsLoRA $r64$ (RetNet)	0.450	0.4822	.596 / .472
LoRA $r16$ (enc-dec encoder)	0.378	0.4071	.396 / .444
Sentence-only ablation (GQA)	0.348	0.3804	.430 / .397
Phase 4 (Sec.: P / R)			
Ens. full+LoRMA (GQA), thr .335	**0.702**	**0.6934**	.702 / .677
Full FT (GQA)	0.693	0.6796	.688 / .664
LoRMA $r16$ (GQA)	0.692	0.6721	.687 / .638
Partial FT top-6 (RetNet)	0.668	0.6611	.659 / .676
KD 6 runs → 8-layer GQA	0.661	0.6533	.671 / .609
AdaLoRA (RetNet)	0.646	0.6411	.675 / .560
BitFit (MLA)	0.650	0.6249	.631 / .609
Houlsby (enc-dec)	0.569	0.5797	.593 / .528

Variant (backbone)	Local	CB	Sec.
Phase 5 (Sec.: micro-F1 / subset acc.)			
Greedy ens. (full, partial, LoRMA-tail)	**0.363**	**0.3635**	.459 / .031
Full FT (GQA)	0.354	0.3588	.450 / .029
All-run ensemble	0.351	0.3533	.452 / .027
KD 6 runs → 8-layer GQA	0.348	0.3472	.442 / .023
LoRMA (GQA), tail chunks only	0.345	0.3461	.438 / .024
BitFit (MLA)	0.331	0.3304	.429 / .020
AdaLoRA (RetNet)	0.296	0.2994	.394 / .012
Houlsby (enc-dec)	0.247	0.2468	.339 / .004
Phase 6 (Sec.: ROUGE-1 / ROUGE-L)			
MBR ens. (LoRA, KD, Pfeiffer)	**0.837**	**0.8350**	.454 / .221
LoRA $r32$ (GQA)	0.836	0.8345	.452 / .219
KD LoRA-GQA → 8-layer GQA	0.835	0.8339	.443 / .225
Pfeiffer (MLA)	0.833	0.8310	.439 / .211
MBR ensemble of 7 runs	0.833	0.8304	.441 / .220
AdaLoRA (MLA)	0.830	0.8284	.433 / .216
QLoRA NF4 (RetNet)	0.828	0.8244	.417 / .210
Houlsby (enc-dec, FiD)	0.822	0.8182	.395 / .207
LoRMA (enc-dec, FiD)	0.821	0.8180	.390 / .205
LoRMA (enc-dec), one chunk	0.821	0.8163	.391 / .204
Phase 7 (Sec.: P@7 / R@7; all + BM25)			
Ensemble of 7 runs	0.406	**0.4076**	.416 / .400
IA^3 (MLA)	0.406	0.4058	.414 / .398
Greedy ens. (partial-MLA, full-GQA)	**0.421**	0.4052	.414 / .397
Partial FT top-6 (MLA)	0.409	0.4034	.412 / .395
LoRMA (enc-dec), random neg.	0.397	0.4028	.411 / .395
QLoRA NF4 (RetNet)	0.395	0.4028	.411 / .395
KD 3 runs → 8-layer GQA	0.403	0.4011	.409 / .393
Full FT (GQA)	0.401	0.3999	.408 / .392
Prompt tuning, 20 tokens (GQA)	0.399	0.3987	.407 / .391
LoRMA (enc-dec), BM25 neg.*	0.395	–	–
BM25 + temporal penalty only*	0.399	–	–

LoRA (boundary errors 594→241), but finished after the deadline. *Phase 3:* context is decisive (the ablation loses 0.12–0.15), and diverse adapters plus class biases edge out the best single run. *Phase 4:* LoRMA matches full fine-tuning (0.692 vs 0.693 honest dev) with 3.6% of the trainable parameters, and averaging both with a tuned threshold wins. *Phase 5:* label-wise attention with per-statute thresholds on full GQA wins. *Phase 6:* LoRA decoders beat the encoder–decoder by ≈0.015 BERTScore despite reading 3.5× less text (summary-token CE 2.38 vs 2.98). *Phase 7:* lexical matching dominates (BM25 with the temporal penalty alone: 0.399); the dev-best greedy ensemble (0.421) fell to 0.405 on test, a sign of over-selection on 118 dev queries.

7 Error Analysis and Discussion

7.1 Phase-wise failure modes

Phase 1. Continuations are fluent and on topic ("It is well settled that…" after a delay-and-laches prompt) but drift into generic boilerplate after 40–60 words, invent correctly formatted citations and loop at low temperature. **Phase 2.** LoRA-GQA's 1,522 hold-out errors are mostly boundary errors with the right label (594), spurious spans (448) and misses (239). DATE and COURT reach 0.87, but WIT ($\leq$0.04) and APP (0.01–0.11) fail because the same name can be party, counsel or witness depending on a cue outside the window; PREC and CASENO produce 42% of false positives. **Phase 3.** Without context, argument and precedent sentences collapse into RatioOfTheDecision (ArgumentPetitioner 135, PrecedentReliedUpon 102, ArgumentRespondent 101 times), because these roles differ mainly in position and surroundings; CL is harder than IT for every model (LoRMA-GQA 0.429 vs 0.493 dev). **Phase 4.** At 0.5 most runs lean to one class (AdaLoRA predicts class 1 for 41.7% of balanced test documents), which threshold tuning repairs; remaining errors are orders outside the last 2,048 tokens and partly-allowed appeals. **Phase 5.**

Models over-predict (5.1–7.1 statutes per document vs 3.9 gold), so subset accuracy is only 0.03. **Phase 6.** Summaries are ≈30% shorter than references (decoders train on ≤880 summary tokens), and models beat the lead-N BERTScore by only 0.009, often omitting the holding. **Phase 7.** F1@7 is 0.29 for queries with 1–3 gold cases and 0.48 for 4–7; pooled chunk vectors likely capture the area of law but not the provisions and party names that decide a citation, which BM25 bigrams do capture.

7.2 What the metrics do not measure

BERTScore and ROUGE reward similarity to one reference, so a summary can score 0.83 while dropping the holding and an invented citation scores like a real one; strict span-F1 counts "Supreme Court of India" as wrong against "Supreme Court"; F1@7 counts a related but uncited precedent as wrong; a firm would need holding review, boundary-tolerant NER and recall@50 (0.79 on dev).

7.3 Limitations and what you would do next

The foundations saw only 200M tokens (≈1.6 per parameter) of a ≈19 GB corpus, so continued pre-training to ≥1B tokens is the most promising step. Context is 2,048 tokens (1,024 positions in the encoder–decoder), so Phases 4–6 read only part of each judgment, there was no human legal evaluation, and no external legal corpus could be used. Composing the phases (step 2) would propagate errors: a mislabelled role or missed entity upstream silently removes evidence downstream, so each stage would need calibrated confidences. Next: (1) partial fine-tuning plus boundary snapping for NER; (2) predicted rhetorical roles to select ratio and order sentences for Phases 4 and 6; (3) a span-corruption objective for the encoder; (4) a Phase-1 cross-encoder re-ranking the PCR top-50; (5) decoding summaries to reference length.

7.4 Surprising findings

Distilled students matched or beat their teachers. In Phase 3 the 8-layer GQA student (55% of the parameters, 2.5× faster) scored 0.507 on test, above its best single teacher (0.499); in Phase 6 it kept 99.9% of the teacher's BERTScore at 2.9× the speed. With one teacher (Phase 2) it kept only 80%, so ensemble teachers seem to give smoother targets. **More input did not help** (Sec. 5.3), so representation quality, not input length, is the bottleneck. **Phase 1 did not predict downstream rank:** MLA won Phase 1 and GQA had the lowest perplexity, but GQA was the best backbone in Phases 2 and 4–6, RetNet with AdaLoRA the best single Phase-3 model and MLA the strongest in Phase 7.

8 Conclusion

This project delivered a self-hosted pipeline: four 125M-parameter legal language models trained from scratch, plus from-scratch adaptation methods, distilled students and dev-selected ensembles for six tasks; every model trains and runs on one 16 GB T4 (two were used in parallel). Decoding and length control, dev-tuned decision rules, document context, training more backbone layers for strict NER and a BM25 temporal prior moved the metrics; the PEFT method mattered little (LoRMA matched full fine-tuning in Phase 4). Under a parameter and data-source lock the 200M-token foundation is the binding constraint, so the gains came from task engineering around it rather than from the choice of adapter.

LLM Usage Declaration

Completing this section is mandatory. Inconsistent answers relative to the code and the prose will be treated as an integrity issue.

Q1. Which Large Language Model(s), if any, did you use while completing the competition or writing this report? Give model name and vendor (e.g., none / ChatGPT-4.x / Gemini x / Claude x / Grok / Copilot / other).

Answer: Claude Opus 5.5

Q2. For each row, mark Yes or No and state the purpose in one short phrase. "Yes" is acceptable only for the uses permitted by the capstone document.

Part	Used LLM?	Purpose of using the LLM
Understanding concepts / papers	Yes	Understanding papers and concepts used for building the architecture
Ideation / brainstorming at a high level	No	
Debugging small ancillary snippets	Yes	Often my notebooks hit Value errors and tensor mismatch errors so to fix them used LLMs
Plotting / visualization code	Yes	Generate code for plotting graphs
Grammar / writing style of the report	Yes	Finally fixing latex formatting and grammar
Report structure / section wording	Yes	On how to save space to include the architecture and graphs
Designing the model architecture	No	
Hyperparameter search / tuning	No	
Analysis of results / conclusions	No	
Writing the core training / inference code	No	
Other (specify)	No	

Q3. Did you refer to sources other than lecture slides and an LLM? If yes, list them (papers, blogs, GitHub repositories, textbooks, official starter notebook, evaluation scripts). These must also appear in the bibliography when they influenced the method.

Answer: Yes. The papers listed in the bibliography, in particular LoRMA [18], LoRA [17], AdaLoRA [20], rsLoRA [19], IA^3 [21], QLoRA [22], Houlsby [23] and Pfeiffer [24] adapters, BitFit [25], prompt tuning [26], DeepSeek-V2 (MLA) [11], RetNet [13], T5 [12], CAML (label-wise attention) [28], Fusion-in-Decoder [29], DPR [32], ColBERT [33], InfoNCE [34], BM25 [31] and Hinton et al. (distillation) [27]. The official CS781 starter notebook and the official evaluation scripts for Phases 1–7 [4]. My Assignment 3 code (RoPE, GQA/MLA) as the basis for the Phase-1 attention layers. The Hugging Face `tokenizers` and PyTorch documentation. No external code repositories, datasets or pretrained weights were used.

Q4. Apart from any permitted LLM use, what was *your* contribution? In 8–12 sentences describe your own understanding, architecture decisions, implementation, experiments, and analysis. Name at least two things you tried that failed and what you concluded from them.

Answer: I defined the overall plan and the constraints for every phase: four structurally different Phase-1 architectures with a lossless byte-level BPE, and parameter-efficient methods implemented from scratch rather than with the peft library. I ran all training and inference on Kaggle T4 GPUs across seven phases, managed checkpoint-and-resume over many 12-hour sessions, and made and tracked every Codabench submission. I decided to retrain all four backbones on an equal data mix, so that every backbone shares one tokenizer and data stream, which made the architecture comparison fair and allowed distillation between any pair of models. For every phase I chose which technique ran on which backbone, read the dev results, and decided which runs to submit and ensemble. I verified the official scores by matching each submitted file to the notebook that produced it. One failure: in Phase 2, LoRA, LoRMA and adapter runs on a frozen backbone stayed far below partial fine-tuning (0.317 vs 0.410 local), which told me that exact span boundaries need more of the backbone to be trained. A second failure: in Phase 7, every dense retriever on its own stayed around 0.30, below my tuned BM25, so lexical overlap, not the small backbone's embeddings, carries most of the citation signal; a hybrid with a temporal penalty was the right design. A third: reading more input (four encoder chunks instead of one, or head plus tail instead of tail only) did not help, which suggests the 200M-token foundation, not context length, is the bottleneck.

References

[1] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating text generation with BERT," *International Conference on Learning Representations*, 2020.

[2] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*, pp. 74–81, 2004.

[3] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: a method for automatic evaluation of machine translation," in *Proceedings of ACL*, pp. 311–318, 2002.

[4] CS781 Course Staff, IIT Kanpur, "CS781 capstone: Starter notebook, evaluation scripts and capstone document," 2026. Course material, Exploration-Lab/CS-781-Capstone.

[5] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proceedings of ACL*, pp. 1715–1725, 2016.

[6] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[7] N. Shazeer, "GLU variants improve transformer," *arXiv preprint arXiv:2002.05202*, 2020.

[8] B. Zhang and R. Sennrich, "Root mean square layer normalization," in *Advances in Neural Information Processing Systems*, 2019.

[9] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proceedings of EMNLP*, 2023.

[10] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *arXiv preprint arXiv:2104.09864*, 2021.

[11] DeepSeek-AI, "DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model," *arXiv preprint arXiv:2405.04434*, 2024.

[12] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.

[13] Y. Sun, L. Dong, S. Huang, S. Ma, Y. Xia, J. Xue, J. Wang, and F. Wei, "Retentive network: A successor to transformer for large language models," *arXiv preprint arXiv:2307.08621*, 2023.

[14] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations*, 2019.

[15] N. S. Keskar, B. McCann, L. R. Varshney, C. Xiong, and R. Socher, "CTRL: A conditional transformer language model for controllable generation," *arXiv preprint arXiv:1909.05858*, 2019.

[16] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, "The curious case of neural text degeneration," in *International Conference on Learning Representations*, 2020.

[17] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022.

[18] H. Bihany, S. Patel, and A. Modi, "LoRMA: Low-rank multiplicative adaptation for LLMs," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025.

[19] D. Kalajdzievski, "A rank stabilization scaling factor for fine-tuning with LoRA," *arXiv preprint arXiv:2312.03732*, 2023.

[20] Q. Zhang, M. Chen, A. Bukharin, P. He, Y. Cheng, W. Chen, and T. Zhao, "Adaptive budget allocation for parameter-efficient fine-tuning," in *International Conference on Learning Representations*, 2023.

[21] H. Liu, D. Tam, M. Muqeeth, J. Mohta, T. Huang, M. Bansal, and C. Raffel, "Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning," in *Advances in Neural Information Processing Systems*, 2022.

[22] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems*, 2023.

[23] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-efficient transfer learning for NLP," in *Proceedings of ICML*, pp. 2790–2799, 2019.

[24] J. Pfeiffer, A. Kamath, A. R"ucklé, K. Cho, and I. Gurevych, "AdapterFusion: Non-destructive task composition for transfer learning," in *Proceedings of EACL*, pp. 487–503, 2021.

[25] E. Ben Zaken, Y. Goldberg, and S. Ravfogel, "BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models," in *Proceedings of ACL (Short Papers)*, pp. 1–9, 2022.

[26] B. Lester, R. Al-Rfou, and N. Constant, "The power of scale for parameter-efficient prompt tuning," in *Proceedings of EMNLP*, pp. 3045–3059, 2021.

[27] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[28] J. Mullenbach, S. Wiegreffe, J. Duke, J. Sun, and J. Eisenstein, "Explainable prediction of medical codes from clinical text," in *Proceedings of NAACL-HLT*, pp. 1101–1111, 2018.

[29] G. Izacard and E. Grave, "Leveraging passage retrieval with generative models for open domain question answering," in *Proceedings of EACL*, pp. 874–880, 2021.

[30] S. Kumar and W. Byrne, "Minimum Bayes-risk decoding for statistical machine translation," in *Proceedings of HLT-NAACL*, pp. 169–176, 2004.

[31] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[32] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of EMNLP*, pp. 6769–6781, 2020.

[33] O. Khattab and M. Zaharia, "ColBERT: Efficient and effective passage search via contextualized late interaction over BERT," in *Proceedings of SIGIR*, pp. 39–48, 2020.

[34] A. van den Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," *arXiv preprint arXiv:1807.03748*, 2018.